



SGLang-Omni 每日 PR 学习日报

报告日期：2026-06-07 · 仓库：[sgl-project/sglang-omni](https://github.com/sgl-project/sglang-omni) · 关注主题：Omni / 多模态 / 多阶段推理架构

TTS 语音合成

Attention Backend

多模态音频编码

ASR 并发基准

推理基础设施

关于"过去 24 小时"的说明：严格意义上的过去 24 小时 (2026-06-06 → 2026-06-07) 窗口内没有任何 PR 被合并。仓库最近的活跃合并窗口是 2026-06-04 ~ 2026-06-05，共 15 个 PR 被合并。本日报覆盖这一最近活跃窗口，并遵循"讲解深度优先于覆盖广度"的原则：深入讲透 4 个与 Omni / 多模态 / 多阶段架构强相关的技术 PR，简要交代 1 个基础设施 PR，并将 10 个纯文档 PR 归并为一节集中介绍。

15

窗口内合并 PR

4

深度讲解技术 PR

10

文档类 PR

5

涉及主题分类

本期目录

1. #692 · Higgs TTS 异步解码输出处理优化 — 多阶段 TTS / 异步解码 / GPU↔CPU 数据搬运
2. #691 · LLaDA2-Uni 默认注意力后端设为 FlashInfer — Attention Backend / CUDA Graph 联动
3. #686 · 基于 PyAV 的 mp3 / aac / opus 音频编码支持 — 多模态音频输出 / 编解码
4. #647 · Qwen3-ASR 并发门禁对齐 + 并发扩展基准 — ASR / 并发 / CI 阈值标定
5. #690 · 正式 Docker 镜像发布 (基础设施) — 部署 / 镜像命名
6. 文档集群 (10 个 PR 归并讲解) — Higgs/Voxtral/Qwen3 cookbook、语言覆盖、benchmark 文档
7. 附录：核心概念速查表

1 PR 概览

标题：[Higgs]: Optimize Higgs async decode resolve output handling

编号：#692 | 作者：Ratish1 (COLLABORATOR) | 状态：已合并 (merged)

链接：github.com/sgl-project/sglang-omni/pull/692

一句话概括：在 Higgs TTS 的"异步解码"路径里，把一个原本被白白放到 GPU 上构建的输出张量改为直接在 CPU 上构建，省掉一次无意义的 GPU 操作和后续的 GPU→CPU 同步，让端到端吞吐提升约 13~15%。

2 它解决了什么问题

Higgs 是一个自回归 (autoregressive) TTS 模型：它像写文章一样，一个 token 一个 token 地往外吐"声学 token"，每一步都依赖上一步的结果。为了加速，引擎用了一种叫"异步解码 (async decode)"的技巧——把"算下一步"和"处理上一步的输出"重叠起来跑，让 GPU 不空等。

痛点在于：异步解码启动 (launch) 那一刻，其实已经把"下一步要用的 token" (codebook-0) 提前发布到 GPU 上了。但随后那个"滞后一步 (lagged)"的输出处理逻辑，又多此一举地在 GPU 上重新建了一个张量，而这个张量的唯一用途只是马上被 `.tolist()` 转成 Python 列表做输出处理。

换句话说：花钱 (GPU kernel + 一次设备同步) 造了个东西，造完立刻搬回 CPU 拆掉。纯属浪费。

3 具体做了什么改动

核心文件：`sglang_omni/models/higgs_tts/model_runner.py` (仅此一个文件)。

关键是给内部函数 `_decode_collect_host()` 新增一个参数 `next_token_device`，用它来"分流"两条路径：

- 同步解码路径 (async decode 关闭时)：传入 logits 所在的 GPU 设备 → 输出张量仍建在 GPU 上 (因为下一步真的要用它)。行为保持不变。
- 异步 resolve 路径 (async decode 开启时)：传入 `None` → 输出张量直接建在 CPU 上 (因为它只用来做输出处理，不喂给下一步)。

改动前 → 改动后 (函数签名)

```
- def _decode_collect_host(self, combined_cpu, result, requests) -> None:
+ def _decode_collect_host(
+     self, combined_cpu, result, requests,
+     *,
```

```
+     next_token_device: torch.device | None,    # 新增：决定输出张量建在哪
+ ) -> None:
```

最关键的逻辑分叉

```
# 改动前：无脑把 next_token_ids 建在 GPU (logits 所在设备) 上
result.next_token_ids = torch.tensor(
    cb0_per_row, dtype=torch.long,
    device=result.logits_output.next_token_logits.device,    # 总是 GPU
)

# 改动后：按调用方意图分流
if next_token_device is None:    # 异步 resolve：只做输出处理
    result.next_token_ids = torch.tensor(cb0_per_row, dtype=torch.long)    # 默认 CPU
else:    # 同步解码：要喂给下一步
    result.next_token_ids = torch.tensor(
        cb0_per_row, dtype=torch.long, device=next_token_device,    # GPU
    )
```

逐行解释（假设你完全没见过这段）：

- `cb0_per_row` 是一个普通的 Python 列表，里面是每个请求这一步采样出来的 "codebook-0" token（一个整数）。它本来就在 CPU 内存里。
- `torch.tensor(..., device=GPU)` 会触发一次 CPU→GPU 的数据拷贝 + 一个建张量的 CUDA 操作。在异步路径里这完全没必要。
- 改成 `torch.tensor(cb0_per_row, dtype=torch.long)`（不写 device）= 默认建在 CPU 上，零拷贝、零 GPU 同步，紧接着的 `.tolist()` 也更快。

两个调用点对应修改：同步图捕获路径 `_collect_step_outputs_cg` 传入真实 GPU 设备；异步 `post_decode_resolve` 传入 `next_token_device=None`。作者特意强调："采样数学、logits、温度/top-p/top-k、停止条件、流式发射、输出码累积——全部不变"，所以这是一次纯粹的"搬运优化"，不动模型语义。

实测收益（Seed-TTS EN 全量，concurrency=8，1088 请求，异步解码开启）

指标	Main 基线	本 PR	变化
平均延迟 (s)	0.891	0.828	-7.1%
P95 延迟 (s)	1.263	1.181	-6.5%
RTF 均值 (越低越好)	0.2073	0.1940	-6.4%
音频吞吐 (s/s)	37.100	41.965	+13.1%

请求吞吐 (req/s)	8.400	9.645	+14.8%
WER 词错率	1.16%	1.11%	-0.05pp (噪声内)

4 涉及的知识

概念：自回归解码 (Autoregressive Decode)

它是什么 → 模型一次只生成一个 token，并把刚生成的 token 接到输入末尾，作为生成下一个 token 的依据，循环往复。

为什么需要它 → 语言/语音生成本质是“根据前文预测下一个单位”，必须顺序进行。这也是 LLM/TTS 推理慢的根本原因：N 个 token 就要跑 N 次前向。

本 PR 里 → Higgs 每个解码步都要产生一个 token 喂给下一步；正因为“下一步要用”，同步路径才必须把它放 GPU；而异步路径里这个职责已被提前完成，所以可以放 CPU。

概念：同步 vs 异步解码 (Async Decode / 计算与处理重叠)

它是什么 → 一种调度技巧：在 GPU 还在算第 N+1 步时，CPU 同时去“收拾”第 N 步的输出（解码成文本/音频、检查是否结束、流式发送）。两件事在时间上重叠。

为什么需要它 → 如果“算一步 → 等结果搬回 CPU → 处理 → 再算下一步”严格串行，GPU 会在 CPU 处理输出时干等。重叠后 GPU 几乎不停转，吞吐显著提升。类比：餐厅后厨在炒下一道菜时，前厅同时给上一道菜摆盘上桌。

本 PR 里 → 异步路径的输出张量是“滞后一步”专供 CPU 输出处理用的，把它从 GPU 挪到 CPU，正好消除了重叠管线里一段多余的 GPU↔CPU 往返。

概念：Device-to-Host 同步 (D2H) 与设备张量

它是什么 → GPU 显存和 CPU 内存是两块独立内存。 `tensor.cpu()` / 在 GPU 建张量再读它，都会触发跨内存拷贝；而读 GPU 数据往往还要“同步”（CPU 停下来等 GPU 把活干完），这叫 D2H 同步。

为什么需要它 → 同步是性能杀手：它打断了 CPU 和 GPU 的并行流水。高性能推理的一条铁律就是“尽量减少不必要的 D2H 同步”。

本 PR 里 → 原代码在 GPU 上建 `next_token_ids` 紧接着要 `.tolist()`（隐含一次 D2H）；改到 CPU 后，这条数据从头到尾不碰 GPU，省掉了同步点。

概念：Codebook (码本) 与 RVQ 声学 token

它是什么 → 现代神经 TTS 不直接生成波形，而是生成离散的“声学 token”。一段音频常被多个码本 (codebook-0/1/2...) 并行表示 (残差矢量量化 RVQ)，codebook-0 是最粗、最重要的那层。

为什么需要它 → 把连续音频"离散化"成 token，TTS 就能复用 LLM 那套自回归生成 + 采样机制；最后再用声码器（vocoder）把 token 还原成波形。

本 PR 里 → `cb0_per_row` 就是每个请求这一步的 codebook-0 token；异步 launch 阶段它已被发布到 GPU 给下一步用，所以 resolve 阶段无需再在 GPU 上重建。

概念：Model Runner（模型执行器）

它是什么 → SGLang 架构里真正"把一个 batch 喂进模型、跑前向、采样、收集输出"的组件。Scheduler 决定"这一步跑哪些请求"，Model Runner 负责"具体怎么跑"。

为什么需要它 → 把"调度策略"和"模型执行细节"解耦，便于针对不同模型（这里是 `HiggsTTSTModelRunner`）做定制化的前向/解码优化。

本 PR 里 → 所有改动都在 `HiggsTTSTModelRunner` 内部的解码收集环节，对上层 Scheduler 完全透明。

5 我能学到什么 / 延伸思考

- 可复用工程思想 1 —— "数据应该住在它被使用的地方"。不要因为"反正都在 GPU 跑"就把所有张量放 GPU；一份只给 CPU 用的数据，建在 CPU 上能省掉拷贝和同步。
- 可复用工程思想 2 —— 用参数把"两条语义不同的路径"显式分流，而不是写两份重复代码或埋 `if` 猜测意图。`next_token_device: device|None` 让调用方"声明意图"，被调用方按意图执行，既清晰又不破坏旧行为。
- 可复用工程思想 3 —— 严格界定改动边界并写进 PR。作者明确列出"什么没变"（采样、logits、停止条件…），让审阅者一眼确认这是无风险的纯性能优化。
- 前置知识建议：① PyTorch 的 device 语义与 `.cpu()/.cuda()` 同步行为；② CUDA stream 与 kernel launch 的异步性；③ 自回归推理的 prefill/decode 两阶段；④ RVQ / 神经音频 codec（如 EnCodec、SoundStream）的基本原理。

1 PR 概览

标题：[LLaDA2-Uni] Set default attention backend to flashinfer

编号：#691 | 作者：btw616 (COLLABORATOR) | 状态：已合并 (merged)

链接：github.com/sgl-project/sglang-omni/pull/691

一句话概括：给 LLaDA2-Uni 模型显式指定注意力后端为 `flashinfer`，修复一个"之前靠开 CUDA Graph 间接锁定后端、后来 CUDA Graph 被默认关掉后导致后端可能被自动选成不支持的那种"的隐藏 Bug。改动只有 3 行。

2 它解决了什么问题

这是一个非常典型的"隐式依赖被打破"的 Bug，值得细品：

1. LLaDA2-Uni 唯一被完整支持的注意力后端是 `flashinfer`。
2. 之前代码没有显式写明这一点，而是顺带实现的：当时默认开启了 CUDA Graph，而开 CUDA Graph 的副作用恰好会把后端锁定成 `flashinfer`。于是"碰巧能跑"。
3. 后来另一个改动出于其他原因，把 LLaDA2-Uni 的 CUDA Graph 默认关掉了。
4. CUDA Graph 一关，那个"顺带锁定后端"的副作用也没了 → 引擎转而走自动选择逻辑 → 可能选中一个 LLaDA2-Uni 根本不支持的后端 → 出错。

通俗讲：原来有个螺丝是被旁边一块挡板"压住"的，从没人专门拧紧它；有天挡板因别的原因被拆了，螺丝就松了。这个 PR 就是把螺丝正式拧紧——明确写死后端，不再依赖任何副作用。

3 具体做了什么改动

核心文件：`sglang_omni/models/llada2_uni/stages.py`，函数

`create_sglang_dllm_thinker_executor_from_config`。

```
- overrides = {"disable_cuda_graph": True}
+ overrides: dict[str, Any] = {
+     "attention_backend": "flashinfer", # 显式锁定，不再依赖 CUDA Graph 副作用
+     "disable_cuda_graph": True,
+ }
+ overrides.update(server_args_overrides or {})
```

逐行解释：

- `overrides` 是一份"服务器参数覆盖表"：在用默认配置构建 SGLang 后端前，先把这里列的键值强制覆盖进去。

- 原本只覆盖了 `disable_cuda_graph: True` (明确关掉 CUDA Graph)。
- 新增 `attention_backend: "flashinfer"`，把后端从"隐式 → 显式"。
- 紧接着的 `overrides.update(server_args_overrides or {})` 表示：如果用户自己传了覆盖项，仍可以再覆盖掉默认值——既给了安全默认，又保留了灵活性。

注意函数名里的 `dllm` = Diffusion LLM (扩散式语言模型)，`thinker` 指多阶段架构里负责"思考/文本理解"的那一阶段——这正好串到 Omni 多阶段架构的知识点。

4 涉及的知识点

概念：Attention Backend (注意力后端)

它是什么 → Transformer 的核心是"注意力 (attention)"计算。Attention backend 就是具体执行这套计算的底层引擎/算子库，常见有 `flashinfer`、`flash_attention (fa3)`、`triton`、`torch_native` 等。它们算的"结果"等价，但在显存占用、速度、对变长/分页 KV cache 的支持上差别很大。

为什么需要它 → 朴素注意力又慢又费显存。FlashInfer 这类后端用 IO 感知的融合 kernel、对 paged KV cache 和变长序列的专门优化，把推理速度和显存效率拉满。不同模型/硬件对不同后端的支持程度不一，所以可切换。

本 PR 里 → LLaDA2-Uni 只在 `flashinfer` 上被完整验证过，所以把它钉成默认后端，避免引擎自动选到不兼容的实现。

概念：FlashInfer 是什么、凭什么快

它是什么 → 一个专为 LLM 推理服务 (serving) 打造的高性能 attention kernel 库，SGLang/vLLM 等都在用。

为什么需要它 → 推理服务的注意力有三大难点：变长序列、分页 (paged) KV cache、以及 prefill 与 decode 两种截然不同的访存模式。FlashInfer 针对这些场景提供高度优化、可与 CUDA Graph 配合的 kernel。

本 PR 里 → 它是 LLaDA2-Uni 唯一"全功能可用"的后端，因此成为安全默认。

概念：CUDA Graph (CUDA 图)

它是什么 → 一种把"一连串 GPU kernel 启动"录制成一张静态图、之后一键重放的机制。

为什么需要它 → 自回归解码每步要启动几十上百个小 kernel，每次启动都有 CPU 端开销 (launch overhead)。当 kernel 很小时，这些开销甚至超过计算本身。CUDA Graph 把"录一次、重放多次"，几乎消除 launch 开销，对小 batch 解码提速明显。代价是要求形状/控制流固定。

本 PR 里 → 它是这个 Bug 的"导火索"——CUDA Graph 开启时会顺带固定 attention backend；它被默认关闭后副作用消失，才暴露出"后端从未被显式设置"的问题。

概念：多阶段架构里的 Thinker (思考者阶段)

它是什么 → Omni / 多模态模型常被拆成多个"阶段 (stage)"流水线，例如 `thinker` (理解输入、做语义思考、产出中间表示/文本) → `talker / generator` (把中间结果转成语音或其他模态输出)。

为什么需要它 → 不同阶段计算特性差异巨大 (文本理解 vs 音频生成)，拆开后可各自独立调度、独立选后端、独立扩缩容，工程上更可控、性能更好。

本 PR 里 → 改动发生在 `create_sglang_dllm_thinker_executor_from_config` ——即"为 thinker 阶段创建执行器"的工厂函数，说明后端选择是按阶段粒度配置的。

概念：Diffusion LLM (dLLM，扩散式语言模型)

它是什么 → LLaDA2 是扩散式语言模型——不像传统 LLM 那样从左到右逐 token 生成，而是借鉴图像扩散，通过"逐步去噪/逐步揭示被掩码 token"的方式并行细化整段输出。

为什么需要它 → 扩散式生成可能带来更好的并行性与可控性，是当前 LLM 架构的前沿探索方向之一。它的调度路径 (`dllm_scheduler`、`PrefillAdder` 等) 和普通自回归 LLM 不同。

本 PR 里 → 函数名中的 `dllm` 表明 LLaDA2-Uni 走的是扩散式路径，这也解释了它为何对 attention 后端有特殊 (更严格) 的支持要求。

5 我能学到什么 / 延伸思考

- 最大教训 —— 警惕"隐式依赖 / 巧合正确"。一个东西"能跑"不代表"被正确配置"。当某行为依赖另一个不相关功能的副作用时，后者一旦变化就会爆雷。把隐式约束变成显式声明是健壮系统的基本功。
- 设计模式 —— "安全默认 + 可覆盖"。`overrides` 先给死安全值，再 `update(用户覆盖)`，是配置系统的经典写法：默认正确，高级用户仍可定制。
- 3 行改动 ≠ 不重要。真正的功夫在于诊断——看懂"CUDA Graph 关闭 → 后端自动选择 → 选到不支持"这条因果链，比写这 3 行难得多。
- 前置知识建议：① Transformer attention 与 KV cache；② FlashInfer/FlashAttention 的差异；③ CUDA Graph 的工作原理与限制；④ SGLang 的 `server_args` 配置体系与按模型/阶段覆盖机制。

#686

基于 PyAV 的 mp3 / aac / opus 音频编码支持

merged · 多模态 ·
Feature

1 PR 概览

标题：Add support for pyAV-based mp3, aac, opus audio encoding

编号：#686 | 作者：haojin2 (COLLABORATOR) | 状态：已合并 (merged)，替代旧 PR #667

链接：github.com/sgl-project/sglang-omni/pull/686

一句话概括：给 TTS 服务的音频输出新增一条基于 PyAV (FFmpeg 的 Python 绑定) 的编码主路径，把模型生成的原始波形编码成 **mp3 / aac / opus** 三种压缩格式，并自带"采样率不合法就自动重采样"和"编码失败回退到 pydub/WAV"的双保险。

2 它解决了什么问题

TTS 引擎内部生成的是原始浮点波形（一串 -1.0~1.0 的数）。但客户端（浏览器、App、电话系统）需要的是标准音频文件：WAV 太大、不适合传输；实际场景往往要 mp3（通用）、opus（实时语音/WebRTC 首选，低延迟高压缩）、aac（苹果生态/流媒体常用）。

旧实现依赖 pydub 来做 mp3/aac/opus。pydub 的问题是：它本质是"把音频写到临时文件 → 调外部 ffmpeg 命令行 → 再读回来"，依赖外部进程、不够稳、控制粒度粗。而且不同编码器对采样率有硬性要求（比如 Opus 只接受 8k/12k/16k/24k/48k），采样率不对就直接报错。

这个 PR 改用 PyAV——直接在进程内调用 FFmpeg 的 C 库做编码，更快更稳；并加上"采样率不合法 → 自动重采样到最近的合法值"的兜底，避免直接失败。

3 具体做了什么改动

核心文件：`sglang_omni/client/audio.py`（新增编码逻辑）+ `tests/unit_test/client/test_audio.py`（新增 144 行单测）+ 3 个 cookbook 文档更新格式说明。

① 用一张配置表声明每种格式的"规矩"

```
PYAV_ENCODE_CONFIGS = {
    "opus": {
        "container": "ogg",                # opus 装进 ogg 容器
        "codecs": ("libopus", "opus"),    # 优先 libopus, 退而求其次用内置 opus
        "valid_rates": {8000, 12000, 16000, 24000, 48000}, # Opus 只认这几种采样率
    },
    "aac": { "container": "adts", "codecs": ("aac",), "valid_rates": {7350, 8000, ..., 96000}
```

```
"mp3": { "container": "mp3", "codecs": ("libmp3lame","mp3"), "valid_rates": {8000,  
}
```

这是"数据驱动"写法：把三种格式的差异（容器、可用编码器、合法采样率）抽成数据，而不是写三套 if-else。新增格式只要再加一行配置。

② 采样率不合法就线性重采样到最近的合法值

```
def _resample_linear(audio, orig_sr, target_sr):  
    if orig_sr == target_sr:  
        return audio.astype(np.float32, copy=False) # 相等直接返回  
    duration = audio.shape[0] / float(orig_sr) # 原音频时长（秒）  
    new_len = max(int(round(duration * target_sr)), 1) # 目标采样点数  
    old_idx = np.arange(audio.shape[0]) # 原始样点位置 0,1,2,...  
    new_idx = np.linspace(0, audio.shape[0]-1, num=new_len) # 在原区间均匀取 new_len 个  
    return np.interp(new_idx, old_idx, audio).astype(np.float32) # 线性插值出新样点
```

逐行解释：重采样 = 改变"每秒采多少个点"。比如原始 44100Hz，但 Opus 只收 48000Hz，就得把样点序列重新"插值"成 48000Hz 对应的长度。`np.interp` 就是在已知点之间画直线、按新位置取值（线性插值，简单够用）。

③ 真正用 PyAV 编码：建容器 → 选编码器 → 喂帧 → flush

```
def _encode_with_pyav(audio, sample_rate, container_format, codecs, valid_rates):  
    import io, av  
    if sample_rate not in valid_rates: # 采样率非法 → 先重采样  
        nearest = min(valid_rates, key=lambda r: abs(r - sample_rate))  
        audio = _resample_linear(audio, sample_rate, nearest)  
        sample_rate = nearest  
  
    buf = io.BytesIO() # 在内存里造一个"假文件"  
    container = av.open(buf, mode="w", format=container_format)  
  
    stream = None  
    for codec in codecs: # 依次尝试候选编码器  
        try:  
            stream = container.add_stream(codec, rate=sample_rate)  
            break # 成功就停  
        except av.CodecError:  
            continue # 这个编码器没装，试下一个  
    if stream is None:  
        raise RuntimeError("None of the codecs are supported by PyAV.")  
  
    stream.layout = "mono" # 单声道
```

```

frame = av.AudioFrame.from_ndarray(          # numpy 波形 → 音频帧
    audio.reshape(1, -1), format="flt", layout="mono") # flt = float 平面格式
frame.sample_rate = sample_rate

for packet in stream.encode(frame): container.mux(packet) # 编码并写入
for packet in stream.encode(None): container.mux(packet) # 传 None = flush 残余
container.close()
return buf.getvalue()                        # 返回编码后的字节

```

关键点：

- `io.BytesIO()`：在内存里模拟一个文件，PyAV 把编码结果写进它——全程不落盘，比 pydub 调外部 ffmpeg 写临时文件快得多。
- `codecs` 是元组、循环尝试：不同环境装的 FFmpeg 编码器名字可能不同（`libopus` vs 内置 `opus`），逐个试以提高兼容性。
- `format="flt"`：FFmpeg 这些编码器要求输入是“float 平面（planar float）”格式，所以要先把波形整理成这个格式。
- 最后 `stream.encode(None)` 传 `None` 是编码器的固定收尾动作——flush（冲刷）缓冲区里还没吐出的数据。

④ 主入口的优先级 + 回退链

```

if fmt in ("opus", "aac", "mp3"):
    try:
        config = PYAV_ENCODE_CONFIGS[fmt]
        return _encode_with_pyav(arr, sample_rate, **config), mime # ① 首选 PyAV
    except Exception as e:
        logger.warning("PyAV %s encoding failed (%s); falling back to pydub/WAV", ...)
    try:
        from pydub import AudioSegment # ② 回退 pydub
        ...
    except Exception:
        return encode_wav(arr, sample_rate), FORMAT_MIME_TYPES["wav"] # ③ 最终回退 WAV

```

这是一条三级降级链：PyAV → pydub → WAV。任一环境缺依赖或编码失败，都不会让请求彻底崩掉，最坏也能返回一个能播的 WAV。FLAC 仍走 `soundfile` 单独处理。

⑤ 配套单测（节选）

```

def test_encode_audio_opus_resampling():
    sr = 44100 # 故意给一个 Opus 不支持的采样率
    audio = np.sin(2*np.pi*440*np.linspace(0,1,sr)).astype(np.float32) # 1 秒 440Hz 正
    encoded, mime = encode_audio(audio, response_format="opus", sample_rate=sr)
    assert mime == FORMAT_MIME_TYPES["opus"]

```

```
try:    assert encoded.startswith(b'OggS')    # Ogg 容器的魔数（文件头标识）
except: assert encoded.startswith(b'RIFF')    # 若无 PyAV, 回退 WAV(RIFF) 也算通过
```

测试用"文件魔数"验证格式：Ogg 文件以 **OggS** 开头、MP3 以 **ID3**、AAC(ADTS) 帧头以 **0xFFF...** 开头、WAV 以 **RIFF** 开头。这样无需真去解码音频就能确认编码格式正确，且对"没装 PyAV 的环境"也友好（回退到 WAV 同样判定通过）。

4 涉及的知识

概念：PCM / 波形 / 采样率 / 声道

它是什么 → 声音是连续波，计算机用"每秒采 N 次、每次记一个幅度值"来表示，这就是 PCM。N 就是**采样率**（如 24000Hz = 每秒 24000 个点）；几路并行就是几个**声道**（mono=单声道）。

为什么需要它 → 这是音频的"未压缩原始形态"。TTS 模型输出就是这种浮点波形，是一切编码的起点。

本 PR 里 → 输入的 **audio** 就是 numpy 浮点 PCM；采样率决定了能否直接喂给某个编码器，否则要重采样。

概念：编解码器(codec) vs 容器(container)

它是什么 → 这是两件常被混淆的事。**codec**（如 opus/aac/mp3）决定"音频数据怎么被压缩"；**container**（如 ogg/adts/mp3 文件）决定"压缩后的数据 + 元信息怎么打包成一个文件"。

为什么需要它 → 同一种 codec 可装进不同 container（opus 常装 ogg）。分清两者才能正确生成可播放的文件。

本 PR 里 → 配置表把 **codec** 和 **container** 分开声明：opus→ogg、aac→adts、mp3→mp3，正是这层区分的体现。

概念：PyAV / FFmpeg 绑定（进程内编码）

它是什么 → FFmpeg 是事实标准的音视频处理库（C 写的）。PyAV 是它的 Python 绑定，让你在 Python 里直接调用 FFmpeg 的 C API，而不是去命令行敲 **ffmpeg**。

为什么需要它 → 进程内调用避免了"起子进程 + 读写临时文件"的开销和不确定性，速度更快、错误更可控、便于流式处理。

本 PR 里 → 用 PyAV 在内存 **BytesIO** 里完成编码，取代旧的 pydub（pydub 走外部 ffmpeg 命令行）。

概念：重采样（Resampling）与线性插值

它是什么 → 把音频从一个采样率转换到另一个采样率。本质是"在已知样点之间，按新的时间网格重新取值"。线性插值是最简单的取值方式（两点连线取中间值）。

为什么需要它 → 各编码器只接受特定采样率集合；输入不在集合里就必须先转换，否则编码报错。

本 PR 里 → `_resample_linear` 把非法采样率（如 44100/12345）转到最近的合法值（如 48000），保证编码不失败。注意：线性插值质量一般，工程上“够用且无依赖”，追求音质会用 `librosa/soxr` 等更好的算法。

概念：优雅降级（Graceful Degradation）与文件魔数

它是什么 → 降级 = 首选方案失败时自动退到次选，保证服务“不挂只变差”。魔数 = 文件开头的固定字节标识（OggS / ID3 / RIFF），用来快速判别文件类型。

为什么需要它 → 生产环境依赖（FFmpeg 编码器、pydub）不一定都装；降级链保证最坏也返回可播 WAV。魔数让测试无需解码即可验证格式。

本 PR 里 → PyAV→pydub→WAV 三级降级 + 用魔数断言（且把“回退 WAV”也判为通过），使代码在各种环境都稳。

5 我能学到什么 / 延伸思考

- 数据驱动设计：用 `PYAV_ENCODE_CONFIGS` 把“格式差异”抽成配置表，把“编码流程”写成统一函数。新增格式 = 加一行数据，不改逻辑——开闭原则的朴素实践。
- 边界即可靠性：真正的工程难点不在“正常路径”，而在“采样率非法 / 编码器没装 / 库缺失”等边界。本 PR 用重采样兜底 + 三级降级把边界全堵上。
- 测试要对环境鲁棒：单测用 `try/except` 同时接受“PyAV 成功的结果”和“回退 WAV 的结果”，避免 CI 因缺少可选依赖而误红。
- 多模态 I/O 的通用模式：“模型产出原始张量 → 客户端层负责编码成标准格式”是多模态系统的标准分层。理解它，对接图像/视频输出也是同一套思路。
- 前置知识建议：① 数字音频基础（PCM、采样率、量化、声道）；② codec/container 关系；③ FFmpeg/PyAV 的 stream-encode-mux 流程；④ Opus/AAC/MP3 各自的适用场景（实时通话 vs 流媒体 vs 通用）。

1 PR 概览

标题：[CI] Align Qwen3-ASR stage-1 gate to concurrency=2 and add ASR concurrency-scaling benchmark

编号：#647 | 作者：MelodyyyYin (CONTRIBUTOR) | 状态：已合并 (merged)，对应 issue #646

链接：github.com/sgl-project/sglang-omni/pull/647

一句话概括：修正 Qwen3-ASR 的 CI 速度门禁——它实际以并发=2 运行，但速度阈值还是旧的并发=32 数字（松了 2~5 倍，根本拦不住性能回退）；同时新增一个可复现的"ASR 并发扩展基准"脚本，并把 CI 门禁重构为"直接复用基准代码 + 套阈值"。

2 它解决了什么问题

这是关于"CI 性能门禁失真"的故事，对理解工程质量保障很有价值：

- 门禁是什么：CI 里有一个测试，每次提交都会跑 20 条语音转写，检查两件事——转写准不准（WER 词错率）、跑得够不够快（吞吐/延迟/RTF）。任一项超阈值就让 CI 变红，拦住性能/质量回退。
- 错位在哪：CI 的工作流里设置了 `QWEN3_ASR_CI_CONCURRENCY=2`（实际并发=2 跑），但测试文件里写死的速度阈值却还是早期并发=32 时测出来的数字（吞吐 10.3、延迟均值 0.193...）。
- 后果：在并发=2 时真实表现其实更好（吞吐 16.3、延迟 0.121）。用并发=32 的宽松阈值去卡并发=2 的运行，阈值松了 2~5 倍——即使性能明显退化也照样能过，门禁形同虚设。
- 一个反直觉的发现：issue 里观察到"并发=2 居然比并发=32 还快"。原因：20 条这种极小工作量下，高并发主要测的是"突发排队/尾延迟"，而不是稳态吞吐；ASR 并发扩展的收益只有在大工作量下才体现。而 WER 几乎与并发无关（小差异是run-to-run 噪声）。

3 具体做了什么改动

涉及 4 处：

1. `tests/test_model/test_qwen3_asr_ci.py`（门禁）：把速度阈值重标定到并发=2 的真实数字；把代码里默认并发从 32 改成 2（让本地、CI、阈值标定工具用同一工作量）；WER 阈值不变（与并发无关）。
2. `benchmarks/eval/benchmark_qwen3_asr_concurrency.py`（新增 494 行）：一个可复现的"ASR 并发扫描"脚本。
3. `.claude/skills/tune-ci-thresholds/.../config.yaml`：修正"门禁跑在 32"的过时注释。
4. `benchmarks/README.md`：记录新脚本与"并发扩展只对大负载有用"的结论。

最漂亮的设计：门禁与基准"共用同一条转写/打分代码"

这个 PR 真正的工程亮点不是改数字，而是消除重复。改动前，门禁测试文件里自己手写了一整套：起线程池并发请求、算百分位、算 WER、拼 summary/speed 字典……几百行。基准脚本又是另一套类似逻辑。两份代码各自演化，迟早算法不一致、结果对不上。

改动后，把核心逻辑抽到基准脚本里的两个函数，门禁直接 import 复用：

```
# 门禁测试从基准脚本导入"转写"和"打分"两个共享函数
from benchmarks.eval.benchmark_qwen3_asr_concurrency import (
    build_asr_eval_results, # 打分：算 WER + 速度指标，拼成 summary/speed
    run_asr_transcription, # 转写：对一批样本按指定并发跑一遍 ASR
)

# 门禁现在只剩"启动 router → 跑一遍 → 套阈值"三步
outputs, wall_clock_s = asyncio.run(
    run_asr_transcription(
        seedtts_en_samples, port=..., model_path=..., lang="en",
        concurrency=QWEN3_ASR_CONCURRENCY, warmup=QWEN3_ASR_WARMUP_REQUESTS,
    )
)
results = build_asr_eval_results(seedtts_en_samples, outputs, wall_clock_s, "en", ...)
corpus_wer = results["summary"]["corpus_wer"]
# checks.check(corpus_wer <= 阈值, ...) ...
```

对比一下被删掉的旧门禁代码（节选），就能感受到"复用"省了多少手写逻辑：

```
# 旧代码：门禁自己手搓线程池 + 手算百分位 + 手拼 30+ 个指标键
with concurrent.futures.ThreadPoolExecutor(max_workers=QWEN3_ASR_CONCURRENCY) as ex:
    futures = [ex.submit(_transcribe_sample, s) for s in seedtts_en_samples]
    for f in concurrent.futures.as_completed(futures):
        sample_id, text, latency_s, audio_duration_s = f.result()
        ...

def _percentile(values, percentile): # 连百分位都自己实现一遍
    ordered = sorted(values); rank = (len(ordered)-1)*percentile/100.0
    ...

wer_summary = { "corpus_wer": corpus_wer, "wer_corpus": corpus_wer, ... 20 多个键 ... }
```

基准脚本的核心结构

```
# 1) 用 aiohttp 异步并发把一条音频发到 /v1/audio/transcriptions
def make_asr_send_fn(model_name, api_url, *, lang="en", ...):
    async def send_fn(session, sample):
        # 读 wav → multipart 表单 → POST → 计时 → 算 RTF
        ...
```



```

# 注释：不要发 temperature=0, Qwen3-ASR 在纯贪心下会退化（服务端会兜底成 0.1）
return send_fn

# 2) 用共享的 BenchmarkRunner 控制最大并发，跑一轮，返回（结果，墙钟时间）
async def run_asr_transcription(samples, *, port, concurrency, ...):
    runner = BenchmarkRunner(RunConfig(max_concurrency=concurrency, ...))
    return await runner.run(samples, send_fn), runner.wall_clock_s

# 3) 多并发 × 多重复扫一遍，每档算 mean/min/max，并读 router 的 /workers 看路由是否均衡
async def _sweep(args, samples, concurrencies):
    for concurrency in concurrencies:
        for repeat in range(1, args.repeats+1):
            before = _fetch_worker_snapshot(...) # 跑前快照
            ... 跑一轮 ...
            after = _fetch_worker_snapshot(...) # 跑后快照 → 算每个 worker 分到多少

```

逐点解释：

- 异步并发（aiohttp + asyncio）：用协程而非线程发请求，单线程就能维持大量在途请求，开销小、更贴近真实压测。`concurrency` 控制“同时在途的请求数（fan-out）”。
- WER 用 `jiwer.process_words` 计算：把参考文本和识别文本对齐，统计替换/删除/插入，得出词错率。先做 `normalize_text`（小写、去标点等）再比，避免格式差异冤枉模型。
- RTF（Real-Time Factor）：处理耗时 ÷ 音频时长。<1 表示比实时快。ASR 服务的关键速度指标。
- per-worker 路由均衡：通过对比 `/workers` 接口在跑前/跑后的计数差，看请求是否被路由器均匀分给各个 worker，能暴露负载倾斜问题。
- 多重复取 mean/min/max：性能测量天生有抖动，重复多次取统计量，门禁/标定才稳。

4 涉及的知识点

概念：ASR（自动语音识别）与 WER

它是什么 → ASR = 语音转文字。WER（Word Error Rate，词错率）= (替换+删除+插入的词数) ÷ 参考总词数，是衡量识别准确度的标准指标，越低越好。

为什么需要它 → 需要一个客观、可比较的数字来判断模型/系统改动有没有让识别变差。WER 就是 ASR 界的“考试分数”。

本 PR 里 → 门禁用 corpus WER（整语料合并算）和 per-sample WER（逐条算）双重把关；并指出 WER 与并发基本无关，所以这次只动速度阈值、不动 WER 阈值。

概念：并发 / Fan-out 与 continuous batching

它是什么 → 并发（concurrency / fan-out）= 同时发出并保持在途的请求数。推理服务端会把同一时刻到达的多个请求**动态拼成一个 batch**一起算（continuous batching / 连续批处理），请求随到随入、随完随出。

为什么需要它 → GPU 擅长大批量并行；只有保持足够多在途请求，才能把 batch 喂满、把 GPU 利用率拉高、把吞吐做上去。并发太低则 GPU 吃不饱。

本 PR 里 → 核心洞察就是"并发收益依赖工作量规模"：20 条的小门禁下高并发反而更慢（测的是排队尾延迟），只有大语料才显出发并发的吞吐优势。所以小门禁就该用小并发(=2)并按它标阈值。

概念：RTF（实时率）与延迟百分位

它是什么 → RTF=处理时间/音频时长（<1 即快于实时）。延迟百分位 P95/P99 表示"95%/99% 的请求快于这个值"，刻画尾部体验。

为什么需要它 → 平均值会掩盖"少数请求特别慢"的问题，而用户最难受的恰是尾延迟。P95/P99 才能反映真实服务质量。

本 PR 里 → 门禁与基准都报告 latency mean/p95/p99 + RTF mean/p95，全面卡住速度回退。

概念：CI 性能门禁与阈值标定 (threshold calibration)

它是什么 → 在 CI 里设一个性能/质量红线，超过就让构建失败。阈值要根据"当前真实表现 + 合理余量(slack)"来标定，太松抓不到回退，太紧天天误报。

为什么需要它 → 没有自动门禁，性能回退只能靠人肉发现，往往上线才暴雷。门禁是"性能不退化"的自动保险。

本 PR 里 → 关键修复就是"阈值必须按 CI 实际运行的工作量来标定"——并发=2 跑就用并发=2 的数字，否则门禁失真。还配套了 `tune-ci-thresholds` 工具读同一套指标键。

概念：Router / Worker（路由器-工作者架构）

它是什么 → 服务前面有个 router（路由器），后面挂多个 worker（模型实例/副本，常对应 DP 数据并行的多个分片）。router 负责把进来的请求分发给某个 worker。

为什么需要它 → 单个 GPU/实例吞吐有限，多 worker 横向扩展提升总吞吐；router 做负载均衡，避免某个 worker 忙死、别的闲死。

本 PR 里 → 基准通过读 router 的 `/workers` 快照，统计每个 worker 实际分到多少请求（per-worker routed），用来检查路由是否均衡——这是判断"扩展是否健康"的重要信号。

5 我能学到什么 / 延伸思考

- 门禁要"测你真正运行的东西"。阈值与运行配置错位（并发 32 的阈值卡并发 2 的运行）会让门禁名存实亡。基准/标定/门禁三者必须共用同一工作量与同一套指标定义。
- 单一事实来源（Single Source of Truth）。把"转写 + 打分"逻辑抽成共享函数，门禁直接复用基准代码——既消除几百行重复，又保证"门禁衡量的"和"基准报告的"永远一致。这是本 PR 最值得学的工程手法。

- 性能直觉要分场景："并发越高越快"只在大工作量、GPU 没吃饱时成立；小工作量下高并发反而放大排队尾延迟。做性能优化前先想清楚瓶颈在哪。
- 测量要抗噪：多重重复 + 取 mean/min/max + 区分 corpus/per-sample，才能把"真回退"和"run-to-run 噪声"分开。
- 前置知识建议：① asyncio/aiohttp 异步并发模型；② continuous batching 原理；③ WER 计算与文本归一化；④ 性能基准方法学（warmup、百分位、墙钟 vs 累计时间）；⑤ 数据并行（DP）与负载均衡路由。

基础设施（简要）

#690 正式 Docker 镜像发布

merged · 基础设施 · 文档

作者：zhaochenyang20 (COLLABORATOR) · [PR #690](#)

做了什么：把官方 Docker 镜像的命名空间从个人/旧组织 `lmsys/sglang-omni:dev` 改为正式组织账号 `lmsysorg/sglang-omni:dev`，仅改动 `docs/get_started/installation.md` 两行（`docker pull` 与 `docker run` 各一处）。

为什么需要：之前文档指向的镜像仓库不是正式发布位置。统一到官方组织命名空间 `lmsysorg`，用户 `docker pull` 才能拉到正式维护、持续更新的镜像。这类改动虽小，却直接决定"新用户能否一行命令跑起来"。

知识点速记 · Docker 镜像命名：一个镜像引用形如 `[registry/]namespace/repository:tag`。这里 `lmsysorg` 是组织命名空间、`sglang-omni` 是仓库名、`dev` 是标签。把命名空间从个人迁到组织，是项目"从实验走向正式发布"的标志性动作。SGLang-Omni 推荐用 Docker 部署，因为镜像里预装了 FlashInfer、CUDA、各模型依赖，省去复杂的环境配置。

文档集群 (10 个 PR 归并)

窗口内其余 10 个 PR 均为纯文档/cookbook改动，不含代码逻辑变化。集中说明如下，方便你把握项目"在文档侧关注什么"：

A Higgs Audio v3 TTS 文档大刷新（最密集的主题）

- #673 (SilinMeng0510)：重写 Higgs TTS cookbook——换用两个内置参考音色 (Adam/Alexandra) 做语音克隆/流式示例；细化内联控制 token (Control Token) 使用规则（每个请求是一个"turn"，情感/风格/语速/音高等"投递 token"要放在文本之前，`<|prosody:pause|>`、`<|sfx:...|>` 等保持就地定位）；刷新到最新 checkpoint (GRPO 0528-1703, step 599) 的基准分数。
- #684：让 cookbook 与官方 model card 对齐，新增"支持语言"小节（100 语言、按 WER/CER 分两档），用 model-card 对比表替换评测段，填入 1×H100 实测 Seed-TTS EN 吞吐。
- #685 / #683 / #681 (zhaochenyang20)：刷新 Higgs 性能数据、快修 lint、更新 Higgs Audio cookbook。
- #688：修正语言分档计数 (WER/CER<5 档 82→83，5–10 档 18→17，总数仍 100)，让文字与实际列出的语言一致。
- #701 (PopSoda2002)：README 支持模型表里把 Higgs TTS 指向公开权重 `bosonai/higgs-audio-v3-tts-4b`（旧链接非公开），并把"100+ 语言"改成精确的 102 语言。

B 语言覆盖与 benchmark 文档

- #694 (SilinMeng0510)：精简"支持语言"小节，改为给链接、去掉冗长列表。
- #652 (haojin2)：在 `benchmarks/README.md` 为新加的 benchmark 命令行 flag 补文档。

C 全模型 cookbook 校验（信息量最大的一篇）

- #670 (MelodyyyYin)：系统性校验并更新 TTS/ASR/omni/LLaDA2-Uni 各路 cookbook；为 Fish Audio S2-Pro、Qwen3-ASR、Whisper ASR 补齐缺失的 cookbook；纠正 Higgs 公开语音 schema（公开字段是 `references` / `ref_audio` / `ref_text`，而非 `reference_codes`）；把 Whisper ASR 标为实验性（encoder batch size=1，建议用 `response_format=json`）。

附带一个小型运行时修复：dev 镜像里的 `sglang` 版本已超前于 pin，`PrefillAdder` 的 API 变了（不再接受 `dllm_config`），导致旧 DLLM 调度路径在到达模型前就报错。该 PR 把 `dllm_scheduler.py` 适配到新 `PrefillAdder` API（`prefill_max_requests=1`，从 `adder.can_run_list` 跟踪 staging），从而让 LLaDA2-Uni 的 `/v1/chat/completions` 文本路径恢复可用。

从文档集群能学到什么：

- 一个高质量推理项目，文档与代码同等重要：模型链接是否公开、语言计数是否精确、cookbook 参数是否与真实 schema 一致，直接决定用户能否成功跑通。
- cookbook 要"可复现"：贴出确切的采样参数（temperature/top_k/max_new_tokens）、确切的 checkpoint（step 599）、确切硬件（1×H100），别人才能复刻你的数字。

- "文档驱动的回归发现"：#670 在校验文档时反而发现了 PrefillAdder API 漂移的真实 Bug——认真写/验文档常能逼出代码里的隐藏问题。

附录：核心概念速查表

概念	一句话理解
Scheduler（调度器）	决定"每一步把哪些请求拼进 batch、何时抢占/结束"的大脑；与 Model Runner 解耦。
Model Runner	真正把 batch 喂进模型跑前向、采样、收集输出的执行器。
Tokenizer / Detokenizer Manager	入口把文本/音频转成 token，出口把 token 转回文本/音频；常独立成进程以重叠 CPU 与 GPU 工作。
Continuous Batching	请求随到随入、随完随出地动态拼 batch，最大化 GPU 利用率。
KV Cache	缓存历史 token 的 Key/Value，避免每步重算前文；是 LLM 推理显存大头。
RadixAttention	SGLang 招牌：用前缀树(radix tree)在请求间共享相同前缀的 KV cache，省显存提命中率。
Paged KV Cache	把 KV cache 像操作系统内存分页一样分块管理，减少碎片、支持灵活复用。
Attention Backend	执行注意力计算的底层 kernel 库（flashinfer/fa3/triton...），可切换。
FlashInfer	为推理服务优化的高性能 attention kernel 库，擅长变长序列+paged KV。
CUDA Graph	把一串 GPU kernel 启动录成静态图重放，消除小 batch 解码的 launch 开销。
Tensor Parallelism (TP)	把单个大模型的权重切到多张 GPU 上协同算一层，突破单卡显存/算力限制。
Data Parallelism (DP)	复制多份模型实例(worker)各自处理不同请求，横向扩吞吐；前面用 router 均衡。

Prefill vs Decode	prefill 一次性并行处理整段 prompt；decode 之后逐 token 自回归生成。
Async Decode	让 GPU 算下一步与 CPU 处理上一步输出重叠，减少 GPU 空等。
多阶段架构 (Thinker→Talker)	Omni 模型拆成理解阶段与生成阶段流水线，各自独立调度/选后端/扩缩容。
声学 token / Codebook / RVQ	把音频离散成多层码本 token，让 TTS 复用 LLM 的自回归生成范式，最后用 vocoder 还原波形。
RTF / WER / P95	RTF=处理时间/音频时长(语音速度)；WER=词错率(识别准确度)；P95=尾延迟。

本日报由自动化流程生成 · 数据来源：sgl-project/sglang-omni GitHub PR · 覆盖最近活跃合并窗口
2026-06-04 ~ 2026-06-05 · 生成日期 2026-06-07
讲解深度优先于覆盖广度 — 深入 4 个核心技术 PR，归并 10 个文档 PR。